

Министерство Образования Республики Беларусь  
УО Брестский Государственный Технический Университет  
Кафедра ИИТ

Отчет по лабораторной работе № 2  
По дисциплине: «Современный платформы программирования»  
Специальность: «Программная инженерия»

Выполнил:  
О.А. Козюта  
Студент группы ПО-13  
Проверил:  
А.Д. Кулик  
Преп.-стажёр кафедры ИИТ

Брест 2026

**Цель работы:** закрепить навыки объектно-ориентированного программирования на языке Python

### Задание 1

#### Реализовать простой класс

Требования к выполнению

- Реализовать пользовательский класс по варианту.

Для каждого класса

- Создать атрибуты (поля) классов
- Создать методы классов
- Добавить необходимые свойства и сеттеры (по необходимости)
- Переопределить магические методы `__str__` и `__eq__`

**8) Множество целых чисел переменной мощности** – Предусмотреть возможность пересечения двух множеств, вывода на консоль элементов множества, а также метод, определяющий, принадлежит ли указанное значение множеству. Класс должен содержать методы, позволяющие добавлять и удалять элемент в/из множества. Конструктор должен позволять создавать объекты с начальной инициализацией. Реализацию множества осуществить на базе списка. Переопределить

Код:

```
"""Module implementing IntegerSet class for working with sets of integers."""
```

```
from typing import List, Optional
```

```
class IntegerSet:
```

```
    """A class representing a set of integers."""
```

```
    def __init__(self, initial_elements: Optional[List[int]] = None) -> None:
```

```
        """
```

```
        Initialize IntegerSet with optional initial elements.
```

```
        Args:
```

```
        initial_elements: List of integers to initialize the set with.
```

```
        """
```

```
        if initial_elements is None:
```

```
            self.elements: List[int] = []
```

```
        else:
```

```
            self.elements = []
```

```
            seen = set()
```

```
            for x in initial_elements:
```

```
                if x not in seen:
```

```
                    seen.add(x)
```

```
                    self.elements.append(x)
```

```
    def add(self, element: int) -> None:
```

```
        """Add an element to the set if not already present."""
```

```
        if element not in self.elements:
```

```
            self.elements.append(element)
```

```
    def remove(self, element: int) -> None:
```

```
        """Remove an element from the set if present."""
```

```

        if element in self.elements:
            self.elements.remove(element)

def contains(self, element: int) -> bool:
    """Check if element exists in the set."""
    return element in self.elements

def intersection(self, other: 'IntegerSet') -> 'IntegerSet':
    """Return a new IntegerSet with elements common to both sets."""
    result = IntegerSet()
    for x in self.elements:
        if x in other.elements:
            result.add(x)
    return result

def __str__(self) -> str:
    """Return string representation of the set."""
    if not self.elements:
        return "{}"
    return "{" + ", ".join(str(x) for x in sorted(self.elements)) + "}"

def __eq__(self, other: object) -> bool:
    """Check equality between two IntegerSet objects."""
    if not isinstance(other, IntegerSet):
        return False
    return sorted(self.elements) == sorted(other.elements)

def main() -> None:
    """Main function to demonstrate IntegerSet usage."""
    print("=== Работа с множеством целых чисел ===")

    input1 = input("Введите элементы первого множества через пробел: ")
    nums1 = [int(x) for x in input1.split()]
    set1 = IntegerSet(nums1)

    input2 = input("Введите элементы второго множества через пробел: ")
    nums2 = [int(x) for x in input2.split()]
    set2 = IntegerSet(nums2)

    print(f"Первое множество: {set1}")
    print(f"Второе множество: {set2}")
    print(f"Пересечение: {set1.intersection(set2)}")

    val = int(input("Введите число для проверки в первом множестве: "))
    print(f"Число {val} принадлежит первому множеству? {set1.contains(val)}")

    add_val = int(input("Введите число для добавления в первое множество: "))
    set1.add(add_val)
    print(f"После добавления: {set1}")

```

```

rem_val = int(input("Введите число для удаления из первого множества: "))
set1.remove(rem_val)
print(f"После удаления: {set1}")

print(f"Множества равны? {set1 == set2}")

if __name__ == "__main__":
    main()

```

```

Введите элементы первого множества через пробел: 1 2 3 4 5 7
Введите элементы второго множества через пробел: 2 3 6 8 90 123 23
Первое множество: {1, 2, 3, 4, 5, 7}
Второе множество: {2, 3, 6, 8, 23, 90, 123}
Пересечение: {2, 3}
Введите число для проверки в первом множестве: 2
Число 2 принадлежит первому множеству? True
Введите число для добавления в первое множество: 12
После добавления: {1, 2, 3, 4, 5, 7, 12}
Введите число для удаления из первого множества: 1
После удаления: {2, 3, 4, 5, 7, 12}
Множества равны? False

```

## Задание 2

Построить модель программной системы с применением отношений (обобщения, агрегации, ассоциации, реализации) между классами. Задать атрибуты и методы классов. Реализовать (если необходимо) дополнительные классы. Продемонстрировать работу разработанной системы

8) Система Интернет-магазин. Администратор добавляет информацию о Товаре. Клиент делает и оплачивает Заказ на Товары. Администратор регистрирует Продажу и может занести неплательщиков в «черный список».

Код:

```
from typing import List, Optional
```

```

class Product:
    """Product with name and price."""

    def __init__(self, name: str, price: int) -> None:
        self.name = name
        self.price = price

    def __str__(self) -> str:
        return f"{self.name} ({self.price} руб.)"

class User:
    """Base user class."""

    def __init__(self, name: str) -> None:
        self.name = name

class Customer(User):
    """Customer with balance and blacklist status."""

    def __init__(self, name: str, balance: int = 0) -> None:

```

```
super().__init__(name)
self.balance = balance
self.is_blacklisted = False
```

```
def pay(self, amount: int) -> bool:
    """Deduct amount from balance if enough funds."""
    if self.balance >= amount:
        self.balance -= amount
        return True
    return False
```

```
class Admin(User):
    """Admin can add products and blacklist customers."""

    def add_product(self, catalog: List[Product], product: Product) -> None:
        """Add product to catalog."""
        catalog.append(product)
        print(f"Админ {self.name}: добавлен товар '{product.name}'")

    def blacklist(self, customer: Customer) -> None:
        """Add customer to blacklist."""
        customer.is_blacklisted = True
        print(f"Админ {self.name}: клиент {customer.name} заблокирован")
```

```
class Manager(User):
    """Manager can view sales reports."""

    def view_sales(self, shop: 'Shop') -> None:
        """Display all registered sales."""
        if not shop.sales:
            print("Продаж пока нет")
        else:
            print(f"Менеджер {self.name}: просмотр продаж")
            for i, order in enumerate(shop.sales, 1):
                print(f" {i}. Клиент: {order.customer.name}, Сумма: {order.total_cost()} руб.")
```

```
class Order:
    """Customer order containing products."""

    def __init__(self, customer: Customer) -> None:
        self.customer = customer
        self.products: List[Product] = []

    def add_product(self, product: Product) -> None:
        """Add product to order."""
        self.products.append(product)

    def total_cost(self) -> int:
        """Calculate total cost of all products."""
        return sum(p.price for p in self.products)
```

```

class Shop:
    """Shop with catalog and sales history."""

    def __init__(self, admin: Admin) -> None:
        self.admin = admin
        self.catalog: List[Product] = []
        self.sales: List[Order] = []

    def register_sale(self, order: Order) -> None:
        """Process a sale attempt."""
        if order.customer.is_blacklisted:
            print(f"Отказ: клиент {order.customer.name} в чёрном списке.")
            return

        cost = order.total_cost()
        print(f"Попытка оплаты заказа на {cost} руб.")

        if order.customer.pay(cost):
            self.sales.append(order)
            print(f"Успех: продажа оформлена для {order.customer.name}.")
        else:
            print(f"Ошибка: недостаточно средств у {order.customer.name}.")
            self.admin.blacklist(order.customer)

def main() -> None:
    """Main function to run the shop system."""
    try:
        print("=" * 50)
        print("ИНТЕРНЕТ-МАГАЗИН")
        print("=" * 50)

        admin_name = input("Имя администратора магазина: ")
        admin = Admin(admin_name)
        shop = Shop(admin)

        manager_name = input("Имя менеджера магазина: ")
        manager = Manager(manager_name)

        print("\n--- Добавление товаров (напишите 'стоп' для завершения) ---")
        while True:
            name = input("Название товара: ").strip()
            if name.lower() == "стоп":
                break
            try:
                price = int(input(f"Цена товара '{name}': "))
                if price < 0:
                    print("Цена не может быть отрицательной")
                    continue
                admin.add_product(shop.catalog, Product(name, price))
            
```

```

except ValueError:
    print("Ошибка: введите целое число для цены")

if not shop.catalog:
    print("Каталог пуст → завершение программы")
    return

print("\n--- Оформление заказа ---")
customer_name = input("Имя клиента: ").strip()
try:
    money = int(input(f"Сколько денег у {customer_name}? "))
    if money < 0:
        print("Баланс не может быть отрицательным")
        return
except ValueError:
    print("Ошибка: введите целое число")
    return

customer = Customer(customer_name, money)
order = Order(customer)

print("\nДоступные товары:")
for i, product in enumerate(shop.catalog):
    print(f" {i:2d}. {product}")

user_input = input("\nНомера товаров через пробел (например: 0 2 1): ").strip()
if not user_input:
    print("Заказ пуст")
    return

for part in user_input.split():
    try:
        index = int(part)
        if 0 <= index < len(shop.catalog):
            order.add_product(shop.catalog[index])
        else:
            print(f"Неверный номер: {part} (игнорируется)")
    except ValueError:
        print(f"Некорректный ввод: {part} (игнорируется)")

if not order.products:
    print("В заказ ничего не добавлено")
    return

print("\n--- Результат ---")
shop.register_sale(order)
print(f"\nОстаток у клиента {customer.name}: {customer.balance} руб.")

print("\n--- Отчет менеджера ---")
manager.view_sales(shop)

```

```
except Exception as e:  
    print(f"Произошла ошибка: {e}")
```

```
if __name__ == "__main__":  
    main()
```

Вывод:

```
Название товара: вуы  
Цена товара 'вуы': 43  
Админ кэп: добавлен товар 'вуы'  
Название товара: йцу  
Цена товара 'йцу': 123  
Админ кэп: добавлен товар 'йцу'  
Название товара: стоп  
  
--- Оформление заказа ---  
Имя клиента: кен  
Сколько денег у кен? 123  
  
Доступные товары:  
0. цук (2 руб.)  
1. вуы (43 руб.)  
2. йцу (123 руб.)  
  
Номера товаров через пробел (например: 0 2 1): 2  
  
--- Результат ---  
Попытка оплаты заказа на 123 руб.  
Успех: продажа оформлена для кен.  
  
Остаток у клиента кен: 0 руб.  
  
--- Отчет менеджера ---  
Менеджер кэп2: просмотр продаж  
1. Клиент: кен, Сумма: 123 руб.
```

Вывод: закреплены навыки объектно-ориентированного программирования на языке Python.